

National Sun Yat-sen University
Department of Electrical Engineering

114-2 Practical Digital System Design

HW4

Traffic Light

Name: Enping, Chung

Student ID: B133015012

Table of Contents

I.Design Process

II.Implementation

III.Testbench Design

IV.Waveform Analysis

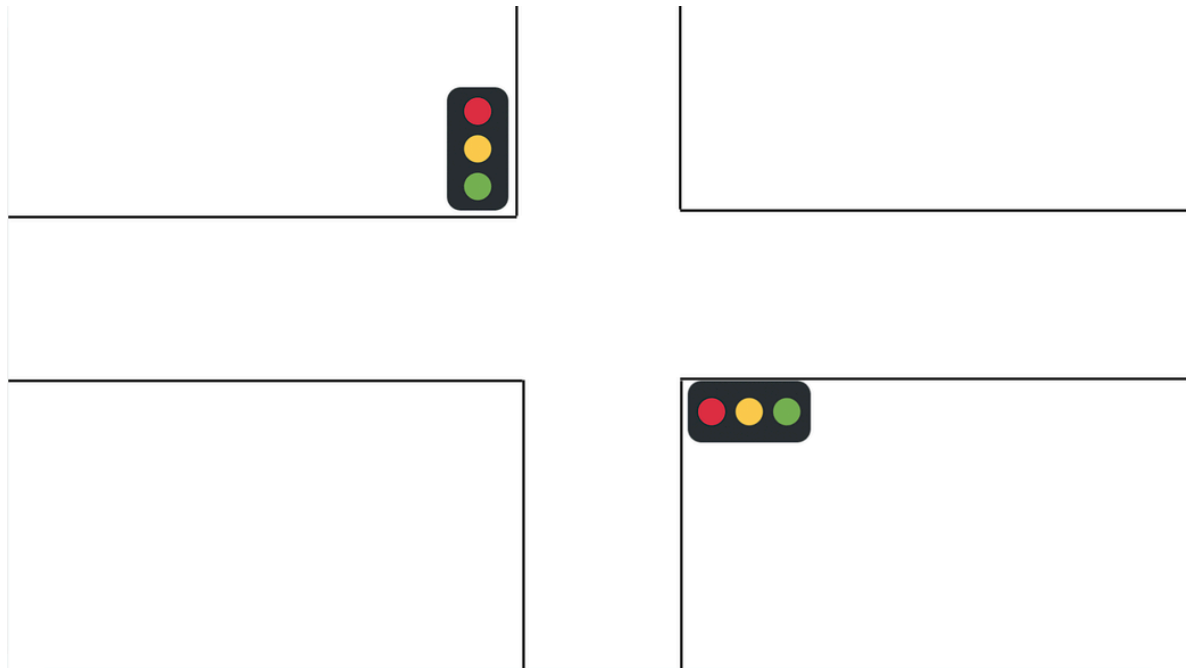
V.Synthesis Procedure

VI.Circuit Analysis

VII.Post Synthesis Waveform
Analysis

VIII.Experience and Reflection

I.Design Process



Goal:

Two roads connected at an intersection need to be controlled by two sets of traffic lights. Each traffic light can display red(stop), yellow(pass), or green(pass). The traffic lights must also take turns flashing yellow lights at night. It is assumed that one of the roads(road A) have more traffic so an extra (Red Light, Red Light) phase is required

Step 1:

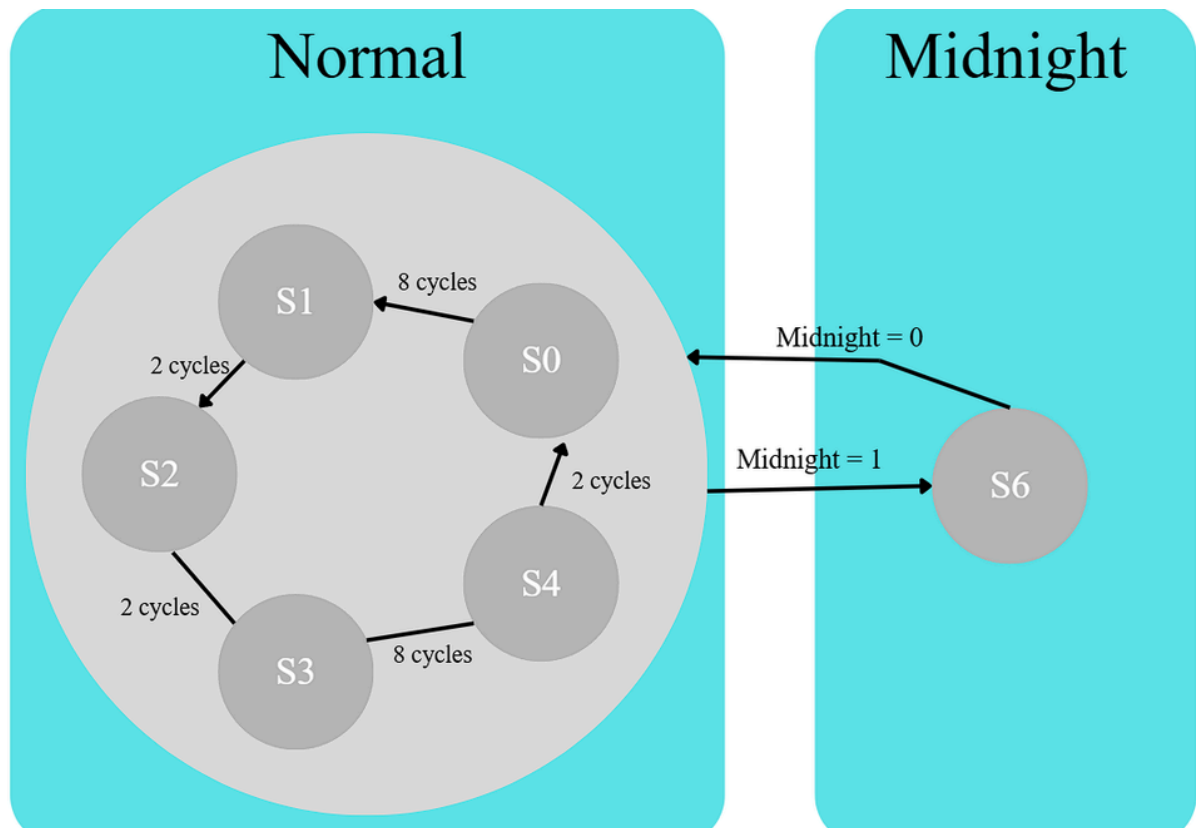
Setting IO ports: The first step in the design process is to define the Module Interface, which establishes how the traffic light controller communicates with external hardware. By specifying the inputs and outputs (I/O), you create a "black box" contract: the inputs capture environmental conditions—such as the system pulse (clk), a master override (reset), and a time-of-day trigger (midnight)—while the outputs provide the control signals for the physical light hardware. Specifically, the design utilizes six boolean outputs (A_G, A_Y, A_R, and B_G, B_Y, B_R) to represent the Green, Yellow, and Red lamps for two perpendicular roads.

Step 2:

Establish state table: With the interface defined, the second step is to map the behavioral logic into a State Table. This table serves as the functional

blueprint for the Finite State Machine (FSM), translating the conceptual timing requirements into specific state transitions and output values. By defining each state (S0 through S6)(S5 is reserved if Road B needs RR phase), we determine exactly which lights are active on Road A and Road B, how long the system remains in that configuration, and what external condition (like the midnight signal) triggers a jump out of the standard sequence.

State	Road A (G/Y/R)	Road B (G/Y/R)	Time (Cycles)	Next State (Midnight=0)	Next State (Midnight=1)
S0	Green	Red	8	S1	S6
S1	Yellow	Red	2	S2	S6
S2	Red	Red	2	S3	S6
S3	Red	Green	8	S4	S6
S4	Red	Yellow	2	S0	S6
S5	Reserved	Reserved	-	S0	S0
S6	Flash Yellow	Flash Yellow	X	S0	S0 (if de-asserted)



This is mainly Moore machine behavior but with some Mealy machine attributes

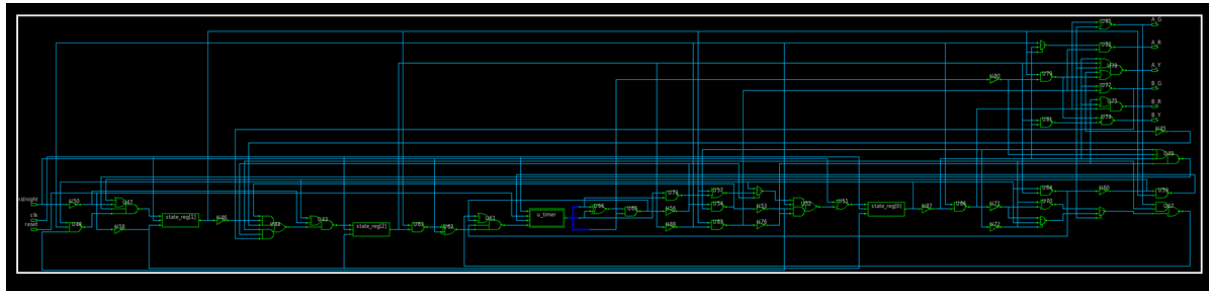
Step 3:

Apply a synchronous counter: since the most important input of our design is the `clk`(states are mostly switched with time), we should design a synchronous counter to use in our. The synchronous counter acts as the system's digital stopwatch, providing the precise timing needed to switch between traffic light phases. Unlike a standard clock that just runs forever, this module is "synchronous," meaning it updates its value only on the rising edge of the master clock to stay perfectly in sync with the controller's logic. It operates under the direct command of the main traffic light module: it increments its count to track how many seconds have passed, but it can be instantly reset to zero by the controller whenever a light changes (such as moving from Green to Yellow). By outputting a simple number—the current "tick" count—it allows the controller to make objective decisions about when to trigger the next transition, ensuring the 8-second and 2-second intervals are accurate and consistent.

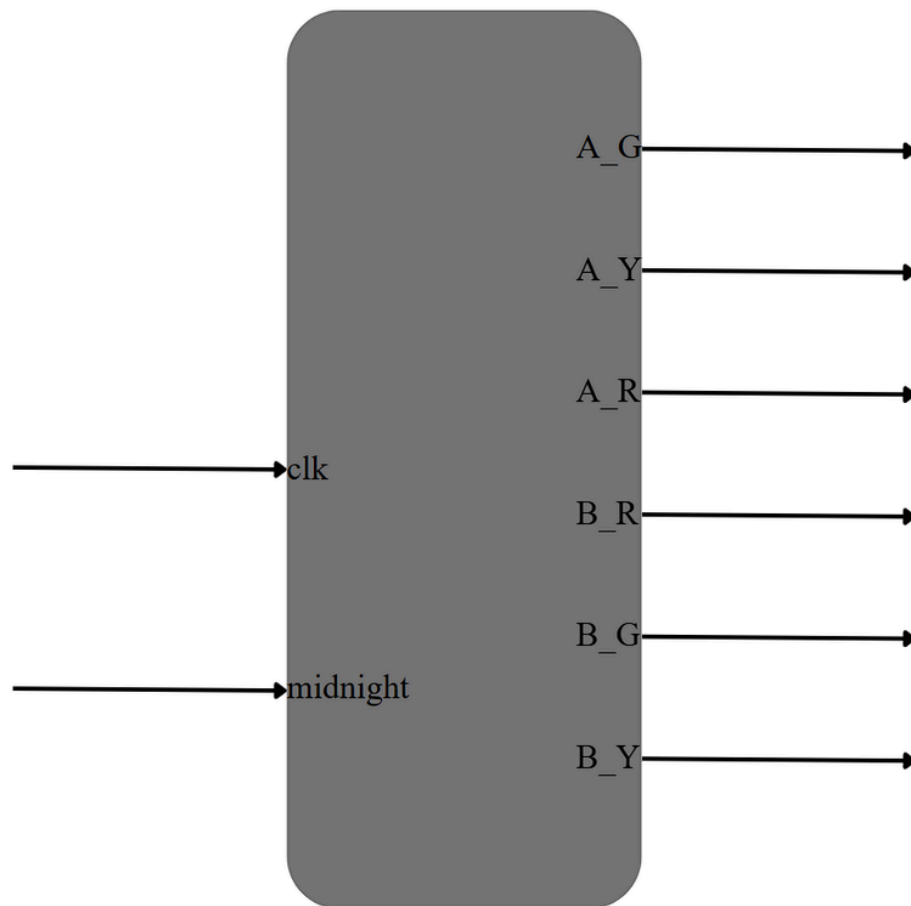
Step 4:

Implementation in Verilog: The circuit will now be written in Verilog and

Sythesized. The Circuit analysis and synthesis process will be seen in later pages. Below is the result in Design Compiler:



II.Implementation



The provided block diagram illustrates the Top-Level Interface of the traffic light controller, serving as a functional "black box" that defines the system's boundary and connectivity. On the left, the control inputs consist of the `clk`, which provides the synchronous timing pulse, and the `midnight` signal, which acts as a global mode-trigger for the caution flash sequence. These inputs drive the internal processing logic—comprised of the synchronous counter and FSM—to generate six actuator outputs on the right. These outputs are grouped into two sets (`A_{G/Y/R}` and `B_{G/Y/R}`), which provide the direct electrical signals required to drive the physical lamps for both roads. This clear separation of I/O allows for modular integration, ensuring that the internal timing logic can be updated without altering the physical wiring interface of the intersection.

The gate level view will be available post synthesis.

III. Testbench Design

The testbench validates the traffic light controller by simulating realistic operating scenarios and checking whether the light outputs match the expected behavior at each moment. It first starts from a clean reset condition, then runs the system in normal daytime mode long enough to cover multiple full traffic cycles, confirming that each phase lasts for the correct duration and appears in the correct order. It then switches to midnight mode to verify the flashing behavior, returns to normal mode to ensure proper recovery, and repeats the midnight transition again to confirm consistency. After that, it performs a reset while the system is active to make sure the controller immediately returns to its safe starting state and restarts correctly after reset release. Throughout the run, each observed light pattern is compared against the expected pattern, all mismatches are counted as errors, and the simulation ends with a clear pass/fail summary based on whether any errors were found.

After testing the traffic light, the testbench then checks the feature of the asyncrous counter checking the noraml counting, reset and hold features.

Terminal Output of unsynthesized design:

VCD info: dumpfile traffic_light.vcd opened for output.														
Time(ns)	Cycle	MIDN	Expected	Actual	Lights	A_G	A_Y	A_R	B_G	B_Y	B_R	Check		
20	0	0	S	0	S 0	100001	1	0	0	0	0	1	PASS	
30	1	0	S	0	S 0	100001	1	0	0	0	0	1	PASS	
40	2	0	S	0	S 0	100001	1	0	0	0	0	1	PASS	
50	3	0	S	0	S 0	100001	1	0	0	0	0	1	PASS	
60	4	0	S	0	S 0	100001	1	0	0	0	0	1	PASS	
70	5	0	S	0	S 0	100001	1	0	0	0	0	1	PASS	
80	6	0	S	0	S 0	100001	1	0	0	0	0	1	PASS	
90	7	0	S	0	S 0	100001	1	0	0	0	0	1	PASS	
100	8	0	S	1	S 1	010001	0	1	0	0	0	1	PASS	
110	9	0	S	1	S 1	010001	0	1	0	0	0	1	PASS	
120	10	0	S	2	S 2	001001	0	0	1	0	0	1	PASS	
130	11	0	S	2	S 2	001001	0	0	1	0	0	1	PASS	
140	12	0	S	3	S 3	001100	0	0	1	1	0	0	PASS	
150	13	0	S	3	S 3	001100	0	0	1	1	0	0	PASS	
160	14	0	S	3	S 3	001100	0	0	1	1	0	0	PASS	
170	15	0	S	3	S 3	001100	0	0	1	1	0	0	PASS	
180	16	0	S	3	S 3	001100	0	0	1	1	0	0	PASS	
190	17	0	S	3	S 3	001100	0	0	1	1	0	0	PASS	
200	18	0	S	3	S 3	001100	0	0	1	1	0	0	PASS	
210	19	0	S	3	S 3	001100	0	0	1	1	0	0	PASS	
220	20	0	S	4	S 4	001010	0	0	1	0	1	0	PASS	
230	21	0	S	4	S 4	001010	0	0	1	0	1	0	PASS	
240	22	0	S	0	S 0	100001	1	0	0	0	0	1	PASS	
250	23	0	S	0	S 0	100001	1	0	0	0	0	1	PASS	
260	24	0	S	0	S 0	100001	1	0	0	0	0	1	PASS	
270	25	0	S	0	S 0	100001	1	0	0	0	0	1	PASS	
280	26	0	S	0	S 0	100001	1	0	0	0	0	1	PASS	
290	27	0	S	0	S 0	100001	1	0	0	0	0	1	PASS	
300	28	0	S	0	S 0	100001	1	0	0	0	0	1	PASS	
310	29	0	S	0	S 0	100001	1	0	0	0	0	1	PASS	

310	29	0	S	0	S	0	100001	1	0	0	0	0	1	PASS
320	30	0	S	1	S	1	010001	0	1	0	0	0	1	PASS
330	31	0	S	1	S	1	010001	0	1	0	0	0	1	PASS
340	32	0	S	2	S	2	001001	0	0	1	0	0	1	PASS
350	33	0	S	2	S	2	001001	0	0	1	0	0	1	PASS
360	34	0	S	3	S	3	001100	0	0	1	1	0	0	PASS
370	35	0	S	3	S	3	001100	0	0	1	1	0	0	PASS
380	36	0	S	3	S	3	001100	0	0	1	1	0	0	PASS
390	37	0	S	3	S	3	001100	0	0	1	1	0	0	PASS
400	38	0	S	3	S	3	001100	0	0	1	1	0	0	PASS
410	39	0	S	3	S	3	001100	0	0	1	1	0	0	PASS
420	40	0	S	3	S	3	001100	0	0	1	1	0	0	PASS
430	41	0	S	3	S	3	001100	0	0	1	1	0	0	PASS
440	42	0	S	4	S	4	001010	0	0	1	0	1	0	PASS
450	43	0	S	4	S	4	001010	0	0	1	0	1	0	PASS
460	MIDN 0	1	S	6	S	6	010000	0	1	0	0	0	0	PASS
470	MIDN 1	1	S	6	S	6	000010	0	0	0	0	1	0	PASS
480	MIDN 2	1	S	6	S	6	010000	0	1	0	0	0	0	PASS
490	MIDN 3	1	S	6	S	6	000010	0	0	0	0	1	0	PASS
500	MIDN 4	1	S	6	S	6	010000	0	1	0	0	0	0	PASS
510	MIDN 5	1	S	6	S	6	000010	0	0	0	0	1	0	PASS

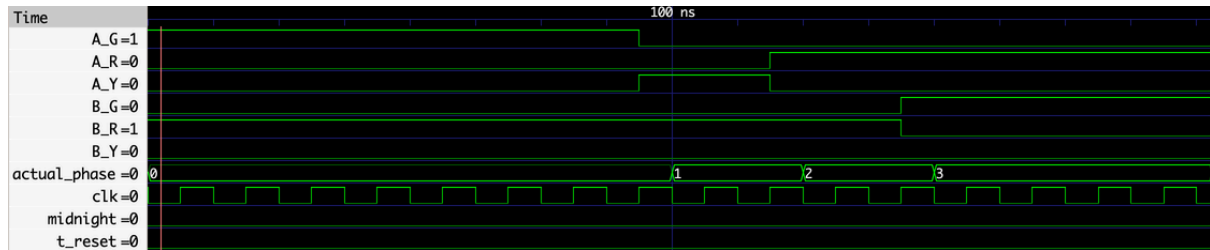
510	MIDN 5	1	S	6	S	6	000010	0	0	0	0	1	0	PASS
520	NORM 0	0	S	0	S	0	100001	1	0	0	0	0	1	PASS
530	NORM 1	0	S	0	S	0	100001	1	0	0	0	0	1	PASS
540	NORM 2	0	S	0	S	0	100001	1	0	0	0	0	1	PASS
550	NORM 3	0	S	0	S	0	100001	1	0	0	0	0	1	PASS
560	NORM 4	0	S	0	S	0	100001	1	0	0	0	0	1	PASS
570	NORM 5	0	S	0	S	0	100001	1	0	0	0	0	1	PASS
580	NORM 6	0	S	0	S	0	100001	1	0	0	0	0	1	PASS
590	NORM 7	0	S	0	S	0	100001	1	0	0	0	0	1	PASS
600	NORM 8	0	S	1	S	1	010001	0	1	0	0	0	1	PASS
610	NORM 9	0	S	1	S	1	010001	0	1	0	0	0	1	PASS
620	NORM10	0	S	2	S	2	001001	0	0	1	0	0	1	PASS
630	NORM11	0	S	2	S	2	001001	0	0	1	0	0	1	PASS
640	NORM12	0	S	3	S	3	001100	0	0	1	1	0	0	PASS
650	NORM13	0	S	3	S	3	001100	0	0	1	1	0	0	PASS
660	NORM14	0	S	3	S	3	001100	0	0	1	1	0	0	PASS
670	NORM15	0	S	3	S	3	001100	0	0	1	1	0	0	PASS
680	NORM16	0	S	3	S	3	001100	0	0	1	1	0	0	PASS
690	NORM17	0	S	3	S	3	001100	0	0	1	1	0	0	PASS
700	NORM18	0	S	3	S	3	001100	0	0	1	1	0	0	PASS
710	NORM19	0	S	3	S	3	001100	0	0	1	1	0	0	PASS
720	NORM20	0	S	4	S	4	001010	0	0	1	0	1	0	PASS
730	NORM21	0	S	4	S	4	001010	0	0	1	0	1	0	PASS
740	MIDN 0	1	S	6	S	6	010000	0	1	0	0	0	0	PASS
750	MIDN 1	1	S	6	S	6	000010	0	0	0	0	1	0	PASS
760	MIDN 2	1	S	6	S	6	010000	0	1	0	0	0	0	PASS
770	MIDN 3	1	S	6	S	6	000010	0	0	0	0	1	0	PASS
780	MIDN 4	1	S	6	S	6	010000	0	1	0	0	0	0	PASS
790	MIDN 5	1	S	6	S	6	000010	0	0	0	0	1	0	PASS
791	RESET	1	S	0	S	0	100001	1	0	0	0	0	1	PASS

Starting test for asynchronous counter

Time(ns)	Step	RST	EN	CLR	Expected	Actual	Check
810	RESET	1	0	0	000	000	PASS
820	COUNT 1	0	1	0	001	001	PASS
830	COUNT 2	0	1	0	010	010	PASS
840	COUNT 3	0	1	0	011	011	PASS
850	COUNT 4	0	1	0	100	100	PASS
860	COUNT 5	0	1	0	101	101	PASS
870	COUNT 6	0	1	0	110	110	PASS
880	COUNT 7	0	1	0	111	111	PASS
890	COUNT 8	0	1	0	000	000	PASS
900	CLEAR	0	1	1	000	000	PASS
910	HOLD	0	0	0	000	000	PASS

[PASS] All tests passed.

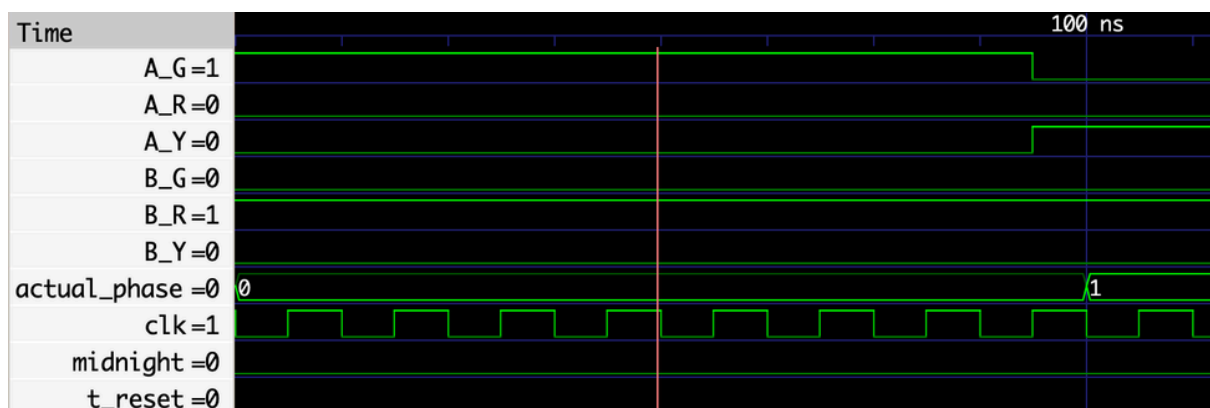
IV.Waveform Analysis



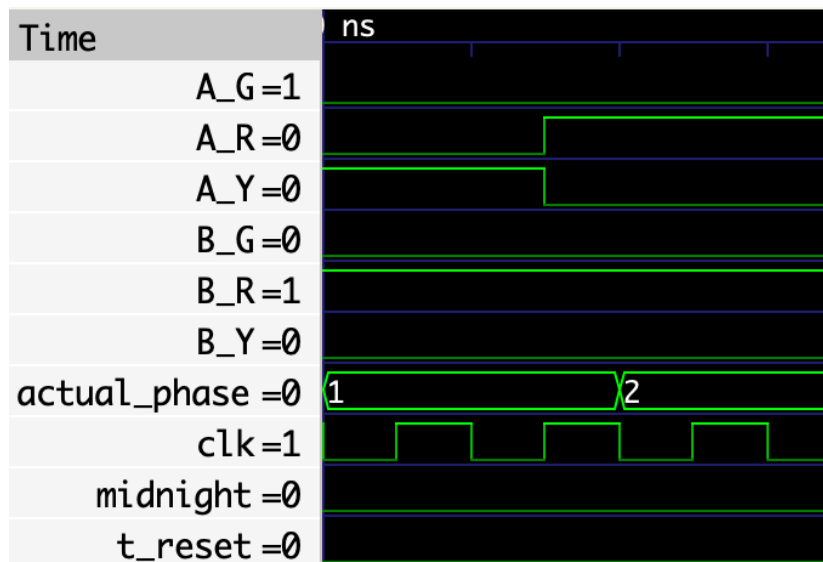
In the above picture, you can see that after 8 clock cycles State goes from 0 to 1, then after 2 more, it goes to State 2, and State 3 after 2 more clock cycles. The following is a walkthrough of all normal phases (midnight = 0) then switch to midnight.

Note that the testbench is checking at the negative edge

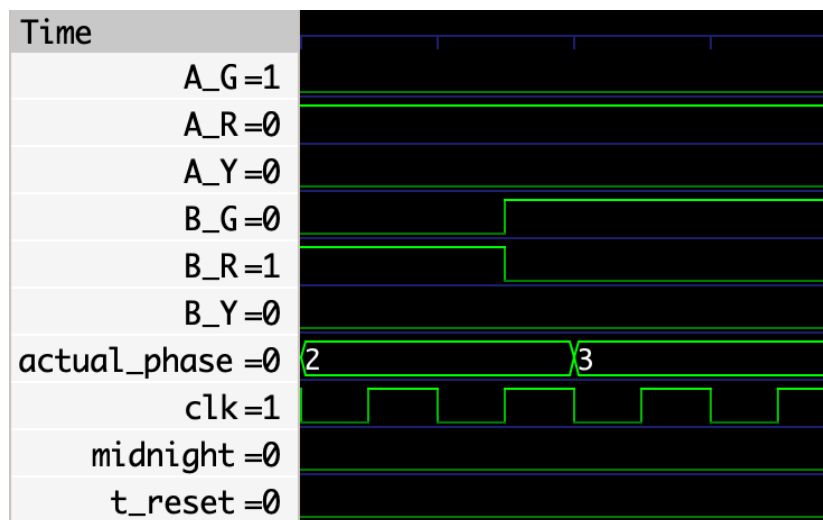
Note that phase is the calculated output decoded from output pins inside the testbench



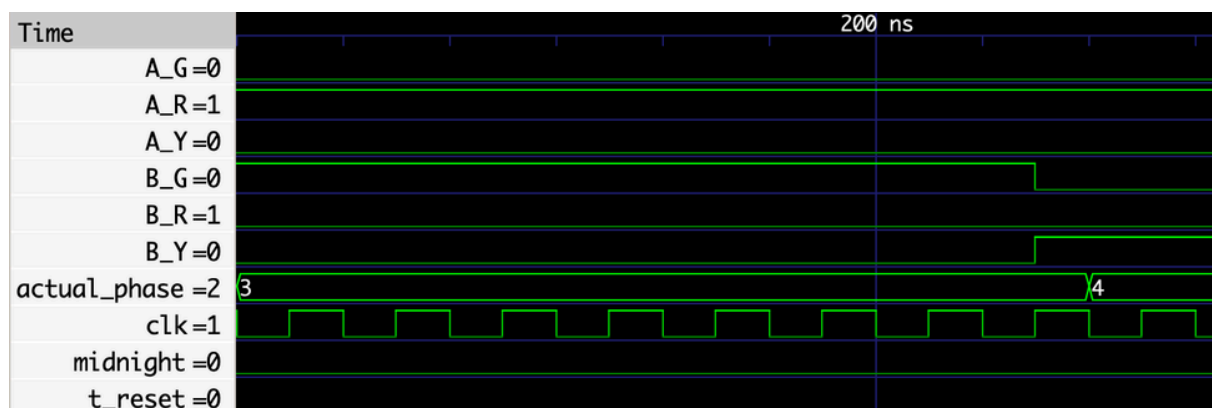
State 0 to State 1 after 8 cycles



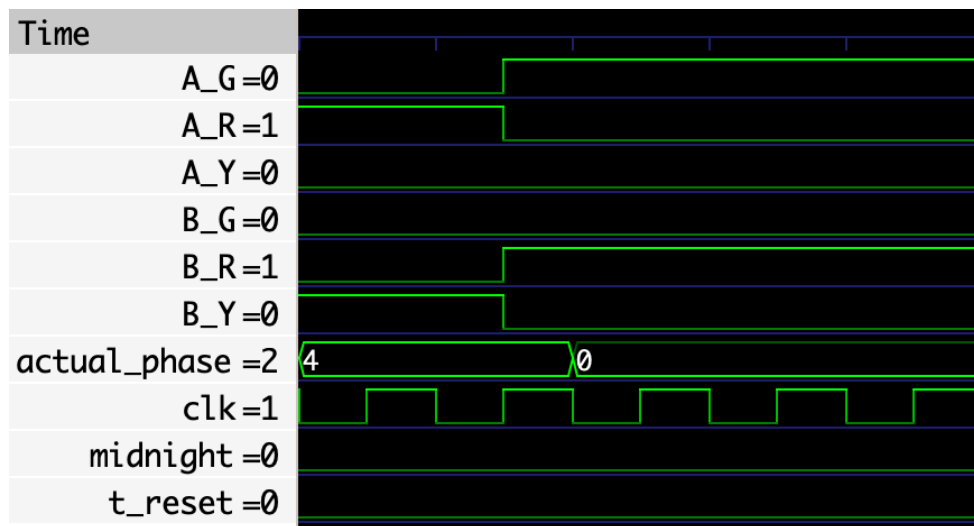
State 1 to State 2 after 2 cycles



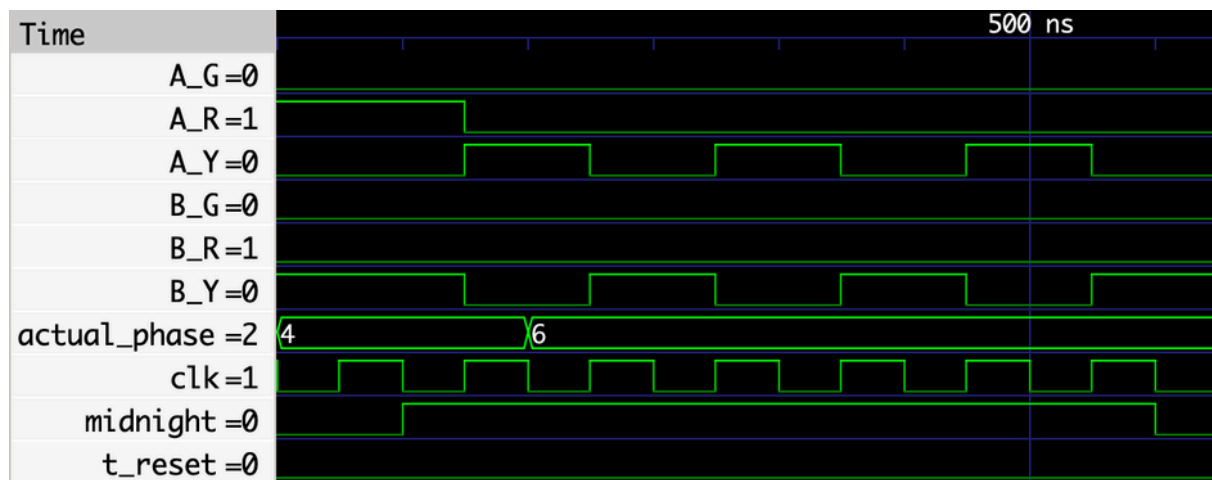
State 2 to State 3 after 2 cycles



State 3 to State 4 after 8 cycles



State 4 to State 0 after 4 cycles



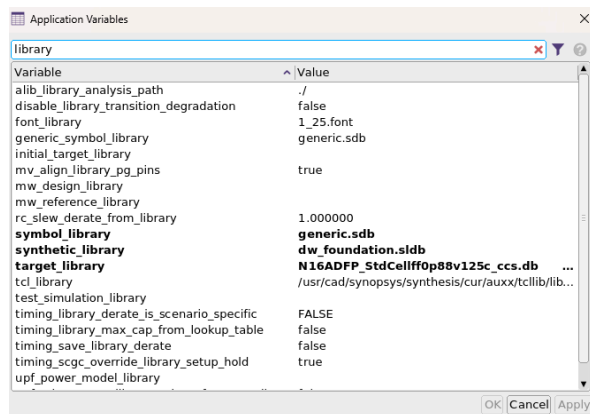
goes to State 6 after midnight = 1

Synthesis Procedure

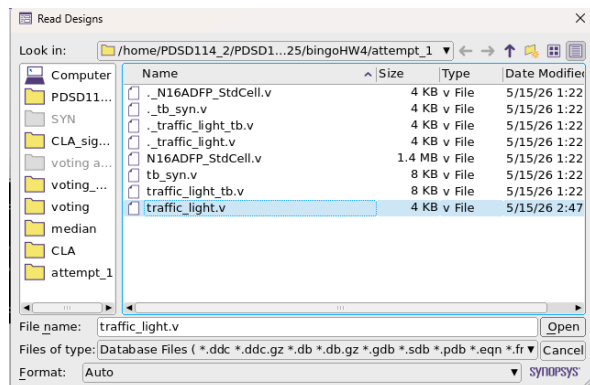
Step 1. Start Program “Design Compiler”: type dv in the terminal

```
soc09 [~/bingoHW4/attempt_1]
-PDSD114_2a25- $dv
```

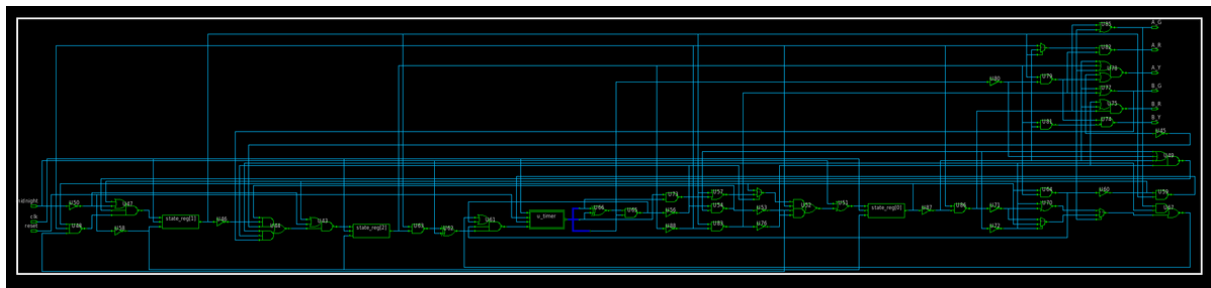
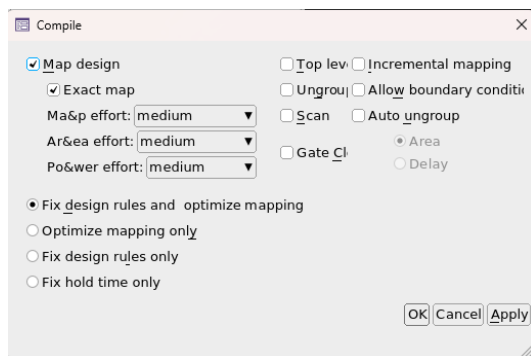
Step 2. Check and choose libraries



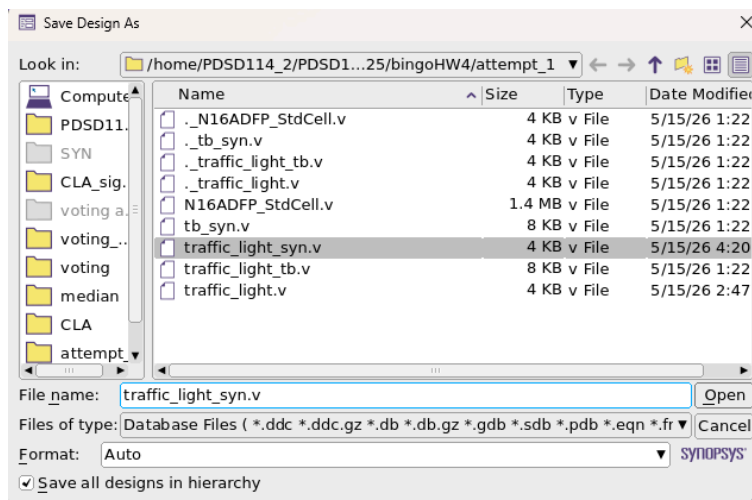
Step 3. Import(Read) File



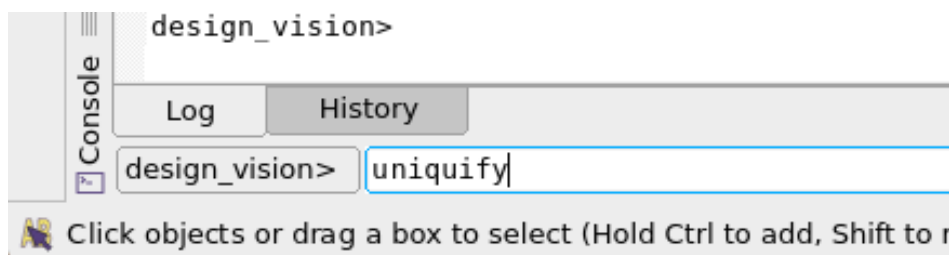
Step 4. Compile Design



Step 5. Save Synthesized File



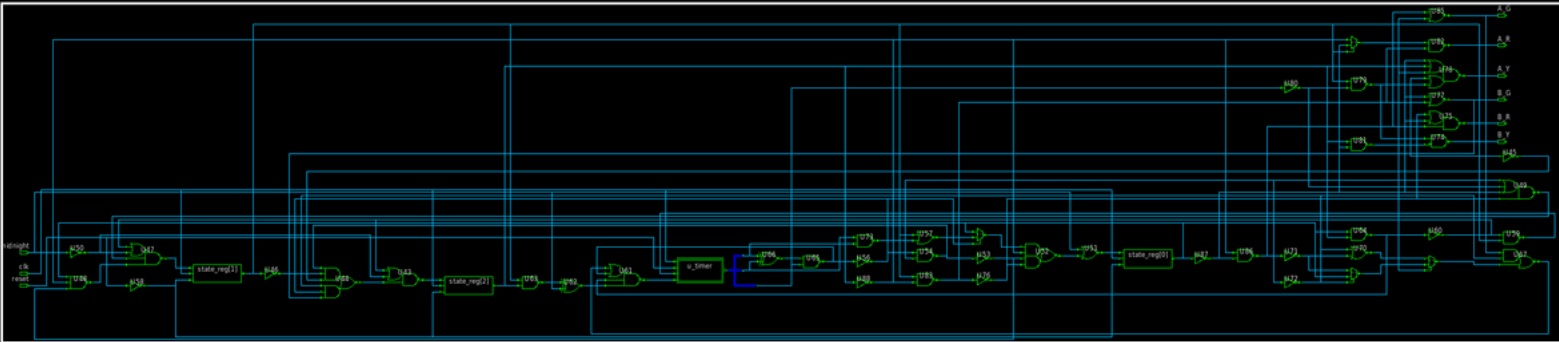
Step 6. Extra command: uniquify (make sure all module names are unique to reduce confusion)



Step 7. Analyze Design (Results Shown in later Slides)

VI.Circuit Analysis

Full Schematic View:



note that u_timer(middle block) is the asynchronous counter module

The traffic light controller is organized as a modular synchronous hardware system with three main parts: state control, timing, and output decoding. The state-control block (an FSM) determines which traffic phase is active, including normal operation phases and midnight flashing mode, and selects the next phase based on reset, midnight mode input, and elapsed time. A synchronous counter provides the timing base for each phase by counting clock cycles; when a phase reaches its required duration, the FSM advances to the next phase and clears the counter so timing restarts for the new phase. The output-decoding block translates the current FSM state into the six lamp control signals for road A and road B (green, yellow, red). Communication between modules is closed-loop: the FSM sends enable/clear control to the counter, the counter returns the current count to the FSM, and the FSM state drives the lamp outputs. This integration gives a fully clocked, predictable, and verifiable architecture that cleanly supports both normal traffic sequencing and midnight flashing behavior.

Area Report:

```
Number of ports:          16
Number of nets:           74
Number of cells:          65
Number of combinational cells: 58
Number of sequential cells: 6
Number of macros/black boxes: 0
Number of buf/inv:        16
Number of references:     17

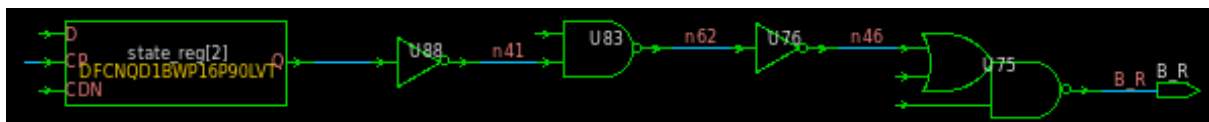
Combinational area:      15.396480
Buf/Inv area:            2.488320
Noncombinational area:   6.220800
Macro/Black Box area:    0.000000
Net Interconnect area:   undefined (Wire load has zero net area)

Total cell area:         21.617281
Total area:              undefined
```

Timing Report:

Point	Incr	Path
-----	-----	-----
state_reg[2]/CP (DFCNQD1BWP16P90LVT)	0.00	0.00 r
state_reg[2]/Q (DFCNQD1BWP16P90LVT)	0.03	0.03 f
U88/ZN (CKND1BWP16P90)	0.01	0.04 r
U83/ZN (ND2D1BWP16P90)	0.01	0.06 f
U76/ZN (CKND1BWP16P90)	0.01	0.07 r
U75/ZN (OAI21D1BWP16P90)	0.01	0.07 f
B_R (out)	0.00	0.07 f
data arrival time		0.07
-----	-----	-----
(Path is unconstrained)		

Critical Path:



Power Report:

```
Global Operating Voltage = 0.88
Power-specific unit information :
  Voltage Units = 1V
  Capacitance Units = 1.000000pf
  Time Units = 1ns
  Dynamic Power Units = 1mW      (derived from V,C,T units)
  Leakage Power Units = 1nW

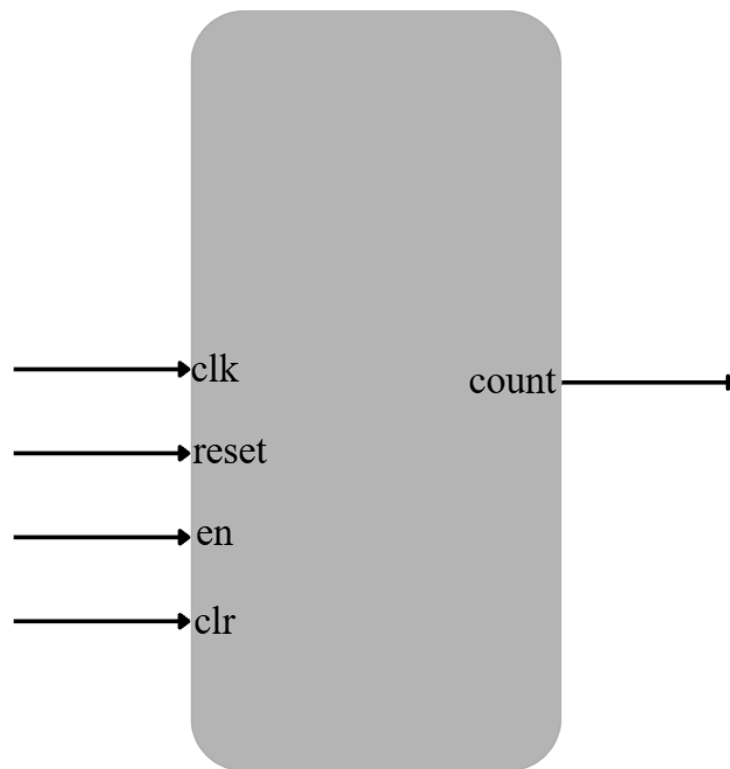
  Cell Internal Power = 2.6912 uW   (65%)
  Net Switching Power = 1.4400 uW   (35%)
-----
Total Dynamic Power   = 4.1312 uW   (100%)

Cell Leakage Power    = 12.2907 uW

Information: report_power power group summary does not include estimated clock tree power. (PWR-789)
```

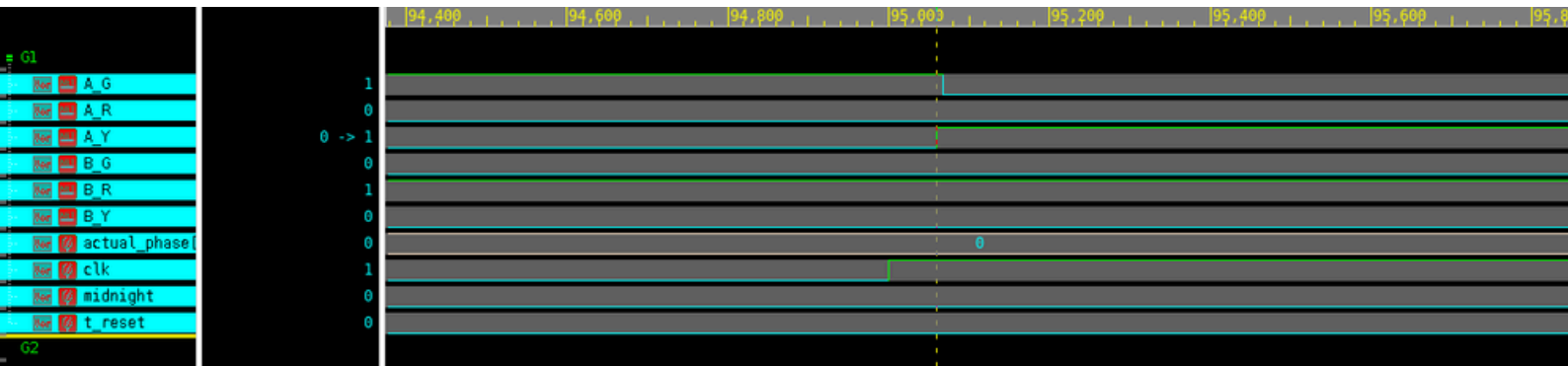
Power Group	Internal Power	Switching Power	Leakage Power	Total Power	(%)	Attrs
io_pad	0.0000	0.0000	0.0000	0.0000	(0.00%)	
memory	0.0000	0.0000	0.0000	0.0000	(0.00%)	
black_box	0.0000	0.0000	0.0000	0.0000	(0.00%)	
clock_network	0.0000	0.0000	0.0000	0.0000	(0.00%)	
register	0.0000	0.0000	0.0000	0.0000	(0.00%)	
sequential	1.4278e-03	1.9296e-04	8.4920e+03	1.0113e-02	(61.58%)	
combinational	1.2634e-03	1.2470e-03	3.7986e+03	6.3090e-03	(38.42%)	
Total	2.6912e-03 mW	1.4400e-03 mW	1.2291e+04 nW	1.6422e-02 mW		

Module-Asynchronous Counter



The synchronous counter acts as the system's digital stopwatch, providing the precise timing needed to switch between traffic light phases. Unlike an asynchronous "ripple" counter where each bit triggers the next, this module is strictly synchronous, meaning all internal flip-flops update simultaneously on the rising edge of the master clock. It operates under the direct command of the main traffic light controller: it increments its value to track elapsed time while providing a 3-bit output that the FSM monitors against target thresholds (like 8 or 2 cycles). Crucially, the module includes a synchronous clear (clr) input, allowing the controller to instantly reset the timer to zero whenever a light changes, ensuring that every new traffic phase begins with a clean and accurate time interval.

VII.Post Synthesis Waveform Analysis

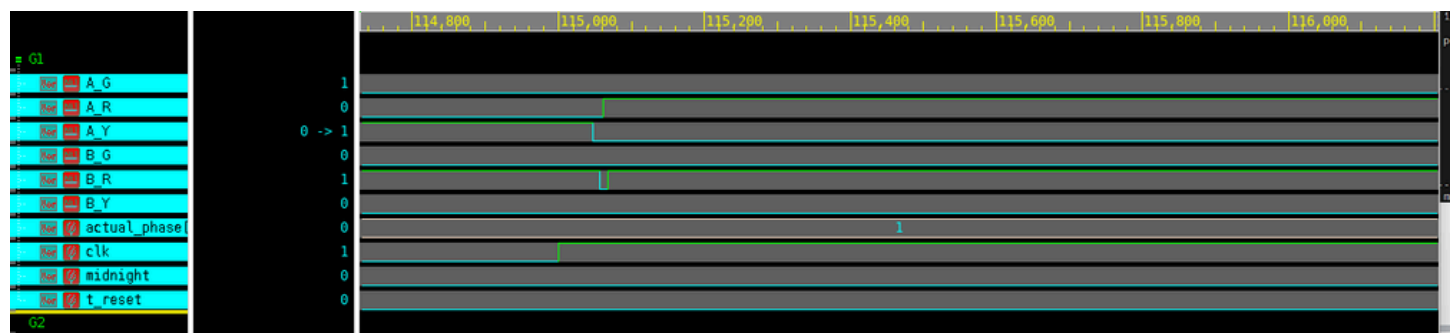


This is a post-synthesis waveform. As you can see, unlike the pre-synthesis design, there is a delay in the combinatorial part of the circuit. In the above picture, you can see that the change from state 0 to state 1 at A_G and A_Y is not aligned with the change in the clock

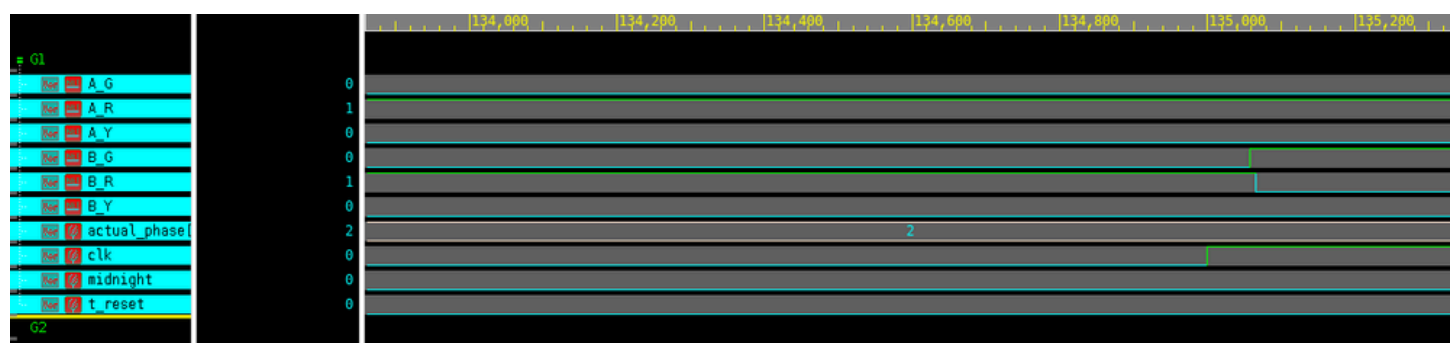
What is the Delay Effect?

Post-synthesis delay effect refers to the physical reality that signals do not move through a circuit instantaneously, as they appear to do in idealized functional simulations. Once Verilog code is mapped to real hardware, every logic gate and routing wire introduces propagation delay, caused by the internal resistance and capacitance of the silicon. This creates a "staggered" arrival of signals where some output pins may toggle nanoseconds before others—even if they are logically part of the same phase—simply because their physical path through the chip is shorter. Furthermore, if multiple inputs to a logic gate change at slightly different times, the circuit can experience glitching (hazards), where an output pin briefly flips to an incorrect value before settling, making the hardware appear out of sync with the intended logical transitions of the state machine.

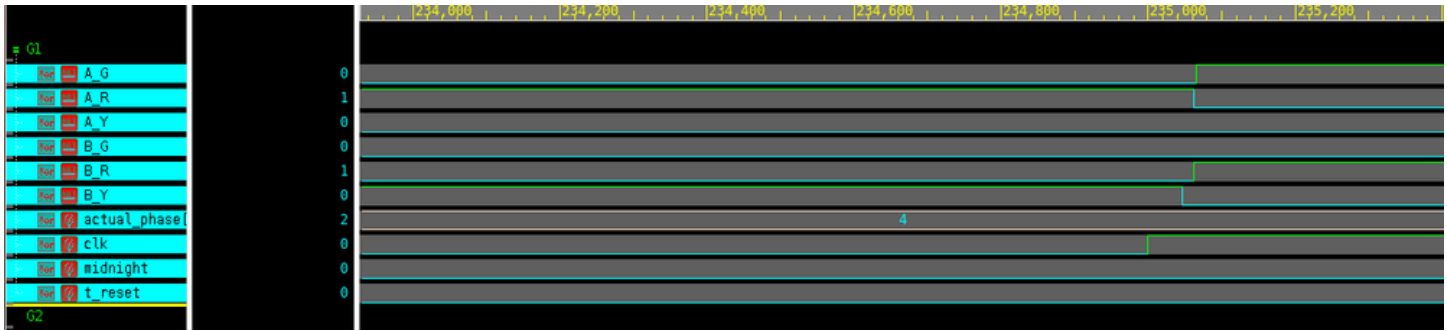
Below are some other state transitions:



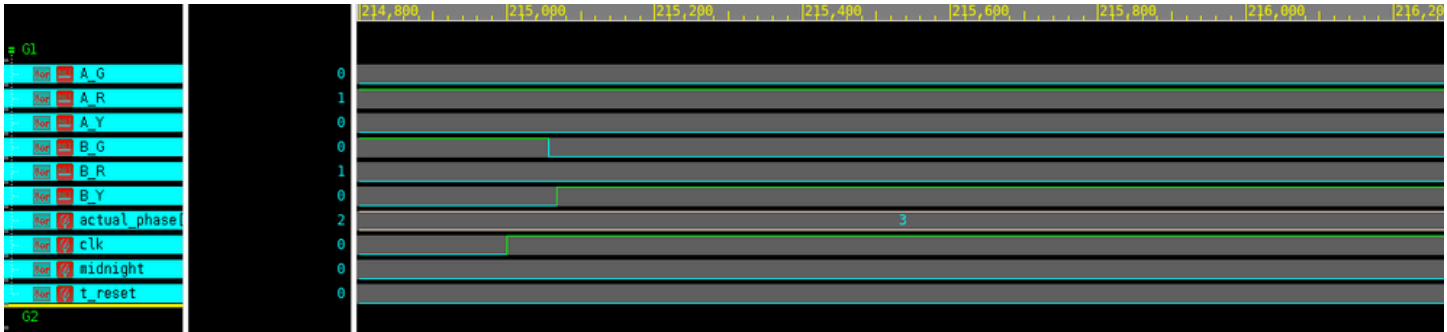
State 1 to State 2



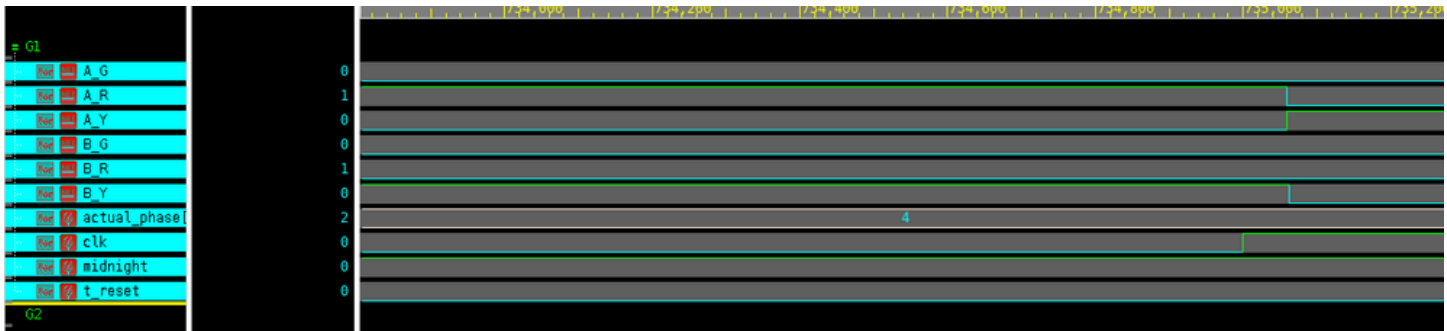
State 2 to State 3



State 3 to State 4



State 4 to State 0



State 4 to State 6

note that midnight = 1

VIII. Experience and Reflection

This project felt like the Final iteration of this course; not only did we need to design a circuit, we had to make up our own features.

Working on the traffic light controller project involved a comprehensive transition from theoretical digital logic to physical hardware implementation. The process began with defining a "black box" interface to establish how the controller communicates with external inputs like the system clock and the midnight trigger. A significant portion of the early work focused on mapping behavioral requirements into a state table, ensuring that each phase (from S0 to S4) met specific timing intervals—such as the 8-cycle green lights and the 2-cycle yellow transitions. A significant portion of the early work focused on mapping behavioral requirements into a state table, ensuring that each phase (from S0 to S4) met specific timing intervals—such as the 8-cycle green lights and the 2-cycle yellow transitions.

The implementation phase moved the design into Verilog, where the three-block RTL coding style was used to separate state registers, next-state logic, and output decoding. The implementation phase moved the design into Verilog, where the three-block RTL coding style was used to separate state registers, next-state logic, and output decoding. The implementation phase moved the design into Verilog, where the three-block RTL coding style was used to separate state registers, next-state logic, and output decoding. Validation was handled through a robust testbench that simulated various scenarios, including normal daytime cycles, transitions to midnight mode, and recovery from reset conditions to ensure the controller always returns to a safe starting state.

Finally, the project addressed the transition to physical silicon through synthesis in Design Compiler. This stage revealed the "Delay Effect," where propagation delays in logic gates and routing wires meant that signals did not move instantaneously as they did in functional simulations. This stage revealed the "Delay Effect," where propagation delays in logic gates and routing wires meant that signals did not move instantaneously as they did in functional simulations. Analyzing these post-synthesis waveforms was crucial for understanding how "staggered" signal arrivals and hardware glitches can make physical outputs appear slightly out of sync with the intended logical transitions. Analyzing these post-synthesis waveforms was crucial for understanding how "staggered" signal arrivals and hardware glitches can make physical outputs appear slightly out of sync with the intended logical transitions.